27
Payment System

Payment systems are the backbone of modern commerce, processing billions of transactions worldwide every day. When you swipe your card at a coffee shop, book a ride-share, or buy something online, you're interacting with a payment system that must be fast, secure, and absolutely reliable. The stakes are high - a single bug can cost millions in lost revenue or create security vulnerabilities that affect thousands of customers.

In system design interviews, payment systems test your understanding of distributed transactions, fault tolerance, and security. The complexity comes from ensuring that money never disappears or duplicates while maintaining high performance and regulatory compliance across multiple jurisdictions.

Step 1 - Understand the Problem and Establish Design Scope

Payment systems can be designed for various purposes - processing customer payments for e-commerce, facilitating peer-to-peer transfers, or enabling merchant payouts. For this tutorial, we'll focus on a general-purpose payment processing system that handles customer payments for e-commerce transactions.

Core payment flows include authorization (checking if a payment can be made), capture (actually taking the money), and settlement (moving funds between accounts). Modern systems must also handle refunds, chargebacks, and various payment methods from credit cards to digital wallets.

Payment refers to the complete transaction between a customer and merchant, including all associated data and metadata.

Payment processing encompasses the technical infrastructure that validates, routes, and executes payment transactions across various networks.

Authorization is the initial check that verifies sufficient funds and fraud risk before holding the customer's money.

Capture is the actual transfer of funds from the customer's account to the merchant's account, typically happening after authorization.

Settlement is the final step where accounts are updated and funds are transferred between banks.

Refund allows merchants to return money to customers for various reasons, including returns or cancellations.

Chargeback is a forced reversal initiated by the customer's bank when there's a dispute or fraud claim.

Payment gateway is the technical interface that connects merchant systems to various payment networks.

Payment processor handles the complex routing, validation, and settlement logic for different payment methods.

Functional requirements

- Accept payments from customers using various methods
- Process refunds when customers request them

Non-functional requirements

- Ensure 99.99% uptime - payment failures directly impact revenue
- Achieve sub-200ms response times for payment authorization
- Maintain PCI DSS compliance for security and data protection

Back of the envelope estimation

For this system design, we'll assume we're processing 1 million transactions per day, which is a reasonable scale for a mid-size e-commerce platform. This translates to roughly 12 transactions per second on average, with peak loads reaching 50-100 transactions per second during business hours.

Step 2 - Propose High-Level Design and Get Buy-In

- Payment Initiation
- Fund Authorization
- Fund Capture
- Settlement
- Refunds

Payment processing involves several key components that work together to ensure secure, reliable transactions. The system must integrate with various external networks while maintaining internal consistency and providing real-time feedback to customers.

Fig 1 - Payment Flow

Payment Initiation

Payment initiation begins when a customer wants to purchase something and chooses their payment method. The merchant's checkout system creates a payment request and sends it to our payment gateway. The gateway validates the request format and routes it to the appropriate processor based on factors like geography, payment method, and merchant configuration.

Fund Authorization

Authorization checks whether the customer has sufficient funds and whether the payment seems legitimate. The processor sends the request through the card network to the customer's bank, which performs various fraud checks and holds the requested amount if approved.

Fund Capture

Capture actually transfers the money from the customer's account to the merchant's account. This typically happens after authorization and may be delayed for hours or days depending on business requirements.

Settlement

Settlement involves the final accounting and actual fund transfers between banks. This happens in batches and involves complex reconciliation processes to ensure all parties receive the correct amounts.

Refunds

Refunds allow merchants to return money to customers for various reasons. The system must track the original transaction and ensure refunds don't exceed the original amount.

Step 3 - Propose High-Level Design and Get Buy-In

Payment Gateway

The payment gateway acts as the entry point for all payment requests. It handles authentication, rate limiting, and initial validation before routing requests to appropriate processors.

Fig 2 - Payment Gateway

Payment Processor

The payment processor handles the complex business logic of payment processing. It maintains connections to various payment networks and implements routing algorithms to optimize success rates and costs.

Fig 3 - Payment Processor

Card Network

Card networks (Visa, MasterCard, American Express) provide the infrastructure that connects merchants and banks. They maintain rules and fraud detection systems that all participants must follow.

Banks

Two banks are involved in every transaction: the merchant's bank (acquiring bank) and the customer's bank (issuing bank). The merchant's bank maintains the merchant's account and receives payments, while the customer's bank manages the customer's account and performs fraud checks.

Data Model

Payment systems require robust data models that can handle various payment methods and maintain complete audit trails.

Payment

- Payment ID
- Amount
- Currency
- Payment method
- Merchant ID
- Customer ID
- Status (pending, authorized, captured, failed, refunded)
- Timestamps for each state change
- Processor response codes
- Fraud scores and risk indicators

Payment Method

- Method ID
- Customer ID
- Type (credit card, debit card, digital wallet, bank transfer)
- Encrypted payment details
- Billing address
- Expiration dates
- Verification status

Transaction Log

- Log ID
- Payment ID
- Event type
- Timestamp
- Processor response
- Error codes and messages
- Retry attempts

API Design

The payment system exposes RESTful APIs for merchants to integrate payment functionality.

Payment Initiation

```
POST /payments
{
  amount: 100.00,
  currency: "USD",
  payment_method: "card_token_xyz",
  merchant_id: "merchant_123",
  customer_id: "customer_456",
  idempotency_key: "order_789_payment_1",
  metadata: {
    order_id: "order_789",
    product_category: "electronics"
  }
}
```

Response

```
{
  payment_id: "pay_abc123",
  status: "pending",
  amount: 100.00,
  currency: "USD",
  created_at: "2024-01-15T10:30:00Z",
  estimated_settlement: "2024-01-17T00:00:00Z"
}
```

Payment Status Query

```
GET /payments/{payment_id}
{
  payment_id: "pay_abc123",
  status: "captured",
  amount: 100.00,
  currency: "USD",
  authorization_code: "auth_def456",
  processor_response: "approved",
  fraud_score: 0.12,
  created_at: "2024-01-15T10:30:00Z",
  authorized_at: "2024-01-15T10:30:05Z",
  captured_at: "2024-01-15T10:30:10Z"
}
```

Refund Request

```
POST /payments/{payment_id}/refunds
{
  amount: 50.00,
  reason: "customer_request",
  idempotency_key: "refund_789_1",
  metadata: {
    ticket_id: "support_123"
  }
}
```

Response

```
{
  refund_id: "ref_xyz789",
  status: "pending",
  amount: 50.00,
  currency: "USD",
  created_at: "2024-01-16T14:20:00Z",
  estimated_settlement: "2024-01-18T00:00:00Z"
}
```

Payment Flow

Let's walk through the complete payment flow from customer interface to fund settlement.

Fig 4 - Payment Flow

Customer Interface

The customer interface must be simple, secure, and trustworthy. Modern payment systems often use hosted checkout pages or embedded widgets to reduce PCI compliance requirements while maintaining control over the user experience.

Payment Gateway Entry

The payment gateway receives the payment request, validates the request format, and performs initial fraud checks. It then routes the request to the appropriate processor based on factors like payment method, geography, and merchant configuration.

Processor Routing

Payment processors maintain connections to various card networks and implement routing algorithms to optimize success rates and costs. They may choose different networks for the same transaction based on real-time performance data.

Card Network Authorization

The card network receives the authorization request and routes it to the customer's issuing bank. The network applies its own fraud rules and maintains statistics on merchant and issuer performance.

Issuer Bank Decision

The issuing bank receives the authorization request and performs various checks including fraud detection, credit limit verification, and account status validation. The bank then approves or declines the transaction.

Response Propagation

The response travels back through the same chain, with each component adding relevant information. The processor may add fraud scores, while the gateway formats the response for the merchant's specific integration.

Fraud Detection

Fraud detection is crucial for payment systems to maintain profitability and customer trust. Modern systems use machine learning algorithms that analyze transaction patterns, customer behavior, and various risk indicators.

Fig 5 - Fraud Detection

Rule-Based Systems

Start with simple rules based on transaction amount, velocity, geography, and merchant category. These provide immediate protection against obvious fraud patterns and are easy to implement and debug.

Machine Learning Models

Advanced systems use machine learning to analyze hundreds of features including customer behavior patterns, device fingerprints, and transaction sequences. Popular approaches include gradient boosting and neural networks.

Real-time Scoring

Fraud scores must be calculated in real-time to enable immediate decisions. The system should provide confidence scores that merchants can use to implement their own risk tolerance.

Feedback Loops

Chargebacks provide ground truth about fraud patterns. The system must incorporate this feedback to improve model accuracy and reduce false positives that frustrate legitimate customers.

Idempotency

Network failures and timeouts are inevitable in distributed systems. Without proper handling, customers may accidentally pay twice when their payment times out and they retry. Idempotency ensures that multiple identical requests produce the same result.

Fig 6 - Idempotency

The idempotency key should be generated by the client and included in all payment requests. The server stores the key and the response for a reasonable period (typically 24 hours) and returns the cached response for any subsequent requests with the same key.

Reconciliation

Even with perfect technical systems, discrepancies can occur due to timing differences, partial failures, or bugs in third-party systems. Reconciliation processes compare our records with those from banks and payment processors to identify and resolve these issues.

Fig 7 - Reconciliation

Automated reconciliation runs daily to match transactions between our system and bank records. The process identifies missing transactions, amount discrepancies, and timing differences that require manual investigation.

Handling payment processing delays

Payment processing can take longer than expected due to network issues, bank system maintenance, or manual review requirements. The system must handle these delays gracefully without losing transactions or creating duplicate payments.

Fig 8 - Handling payment processing delays

Implement exponential backoff with jitter for retry attempts to avoid overwhelming downstream systems. Maintain dead letter queues for payments that fail after maximum retry attempts, enabling manual investigation and resolution.

Communication among internal services

We'll use gRPC for internal service communication between our payment processing components. It provides high performance, strong typing, and built-in support for distributed tracing.

Asynchronous communication

Payment processing involves many asynchronous operations:

- Webhook notifications to merchants about payment status changes
- Fraud model updates based on new chargeback patterns
- Reconciliation jobs that compare our records with bank statements

Fig 9 - Event Driven Architecture

- Message queues ensure reliable delivery even when downstream services are temporarily unavailable.
- Webhook services must implement retry logic with exponential backoff and maintain delivery receipts for debugging.
- Fraud model updates can be batched and deployed during low-traffic periods to minimize system impact.

Fig 10 - Webhook Delivery

Webhook delivery is critical for merchants to know payment status changes. The system must track delivery attempts and provide merchants with tools to monitor and debug webhook failures.

Security and data protection

Payment systems handle sensitive financial data and must implement comprehensive security measures to protect customer information and prevent fraud.

Fig 11 - Data Protection

Key security measures include:

- Encrypt all data in transit using TLS 1.3 and at rest using AES-256
- Tokenize sensitive card data to reduce PCI compliance scope
- Implement strong access controls with multi-factor authentication
- Maintain comprehensive audit logs for all payment operations
- Regular security assessments and penetration testing

Scaling

As payment volume grows, the system must maintain performance while ensuring reliability. Key scaling challenges include database performance, network latency, and third-party integration limits.

Fig 12 - Scaling

- Database sharding by merchant or customer ID distributes load across multiple servers.
- Caching frequently accessed merchant configurations and fraud models reduces database load.
- Message queues decouple payment processing from downstream notifications and analytics.
- Monitoring and alerting must track payment success rates, latency, and error rates across all components.

Step 4 - Wrap Up

Building a payment system requires balancing security, reliability, and performance while navigating complex regulatory requirements. The key to success is starting with a solid foundation that can handle the core payment flows reliably, then adding sophisticated features like fraud detection and reconciliation.

Payment systems are critical infrastructure that must maintain high availability and security. The complexity comes from integrating with various external networks and ensuring consistency across distributed components.

Key takeaways:

- Idempotency is crucial for preventing duplicate payments during network failures
- Reconciliation catches discrepancies that automated systems miss
- Fraud detection must balance security with customer experience
- Regulatory compliance requirements vary by jurisdiction and must be designed into the system from the beginning
- Monitoring and alerting must cover both technical metrics and business outcomes

Remember that payment systems are constantly evolving to support new payment methods, improve fraud detection, and reduce costs. Stay current with industry developments and regulatory changes that may impact your system design.

Chapter Summary

Reference Materials:

Stripe API Docs - https://stripe.com/docs/api/charges/create
Stripe Idempotency - https://stripe.com/docs/api/idempotent_requests
PCI DSS - https://www.pcisecuritystandards.org/
Braintree Fraud Tools - https://www.braintreepayments.com/
Chargebacks - https://www.chargebacks911.com/
Uber Fraud Detection - https://eng.uber.com/michelangelo-machine-learning-platform/
Stripe Fraud Detection - https://stripe.com/docs/radar
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-within-a-distributed-system
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-2
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-3
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-4
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-5
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-6
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-7
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-8
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-9
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-10
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-11
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-12
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-13
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-14
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-15
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-16
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-17
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-18
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-19
Stripe Payment Processing Blog Post - https://stripe.com/blog/payment-processing-at-scale-20